

Dino-QRL: a quantum–classical hybrid agent that plays Chrome-Dino from pixels

A CNN sees the screen; a variational quantum circuit decides the action

Qiskit hackathon — experiments/dino/

Abstract

We build a hybrid agent for a headless Chrome-Dino clone: a convolutional network (CNN) compresses the raw screen into six features, and a six-qubit variational quantum circuit (VQC) — the backbone’s single `build_circuit` — is the Q -function head that chooses the action. The whole stack is one differentiable graph trained end-to-end with Double-QDQN. **This is a demonstration of a hybrid quantum–classical agent on image input, not a quantum-advantage claim.** Dino is trivially solvable by an `if-else` rule (our scripted oracle is one), which is exactly why it is a clean testbed: any behaviour is easy to attribute. On identical footing the quantum head plays *well* (greedy score 1565, clearing dozens of cacti; random 224) while a classical linear head plays *perfectly* (2000) — no advantage, a small quantum cost. Ablations show the quantum structure is nonetheless load-bearing: removing entanglement *or* the two-qubit $\langle ZZ \rangle$ readout collapses the agent to random. The trained circuit reproduces on real Qiskit to 3.4×10^{-7} and was executed on a real IBM quantum computer (`ibm_marrakesh`), which matched the exact simulator’s decision on all 16/16 test frames.

1 Honest framing

The point of Dino is not that it is hard — it is not; a hand-coded `if-else` on obstacle distance plays forever. The point is the *question*: *can a variational quantum circuit serve as the trainable decision head of a deep-RL agent that consumes raw pixels, and how does it compare, on identical footing, to a classical head?* We answer it and report the result faithfully, including where the quantum head is weaker. No quantum advantage is claimed or implied.

2 Architecture

The per-timestep data flow is a closed loop **ENV** $\rightarrow$ **CNN** $\rightarrow$ **quantum circuit** $\rightarrow$ **action**:

1. **Environment.** A pure-pygame Chrome-Dino clone (headless, ~ 2000 steps/s), wrapped to the Gymnasium API. Actions are `Discrete(3) = {RUN, JUMP, DUCK}`. Reward is +0.01 per surviving frame +1.0 per obstacle cleared; the reported *score* is frames survived.
2. **Preprocessing.** Grayscale, crop to the action region (so obstacles are large), resize to 84×84 , stack the last 4 frames $\Rightarrow$ observation $[4, 84, 84]$ uint8. Frame-skip 4 (one decision every four game frames).
3. **CNN encoder (perception).** A Nature-DQN convolutional stack $\text{Conv}(4 \rightarrow 32, 8 \times 8, s4) \rightarrow \text{Conv}(32 \rightarrow 64, 4 \times 4, s2) \rightarrow \text{Conv}(64 \rightarrow 64, 3 \times 3, s1)$ (spatial $84 \rightarrow 20 \rightarrow 9 \rightarrow 7$), flatten ($64 \cdot 7 \cdot 7 = 3136$), $\text{Linear}(3136 \rightarrow 128) \rightarrow \text{Linear}(128 \rightarrow 6) \rightarrow \tanh$. Output: six features $f \in [-1, 1]^6$. **480,294 trainable parameters.**
4. **Encoding into the circuit.** The six features become the circuit’s RY rotation angles: $\theta_q^{\text{enc}} = \lambda_q f_q$, where λ (six params) is a trainable input scaling kept *outside* the circuit.
5. **Quantum circuit (decision).** `build_circuit(n_qubits=6, n_layers=3, reuploading=True)`: each layer re-uploads the data ($\text{RY}(x_q)$), applies trainable $\text{RY}(\theta)/\text{RZ}(\theta)$ on every qubit, then a CNOT entangling ring. Circuit facts: **6 qubits, depth 27, 72 gates** {36 RY, 18 RZ, 18 CX}, **36 trainable weights**. Evaluated on the backbone `torch_sv` exact-autograd statevector backend.

The VQC decision head: 6-qubit data re-uploading ansatz (3 layers)

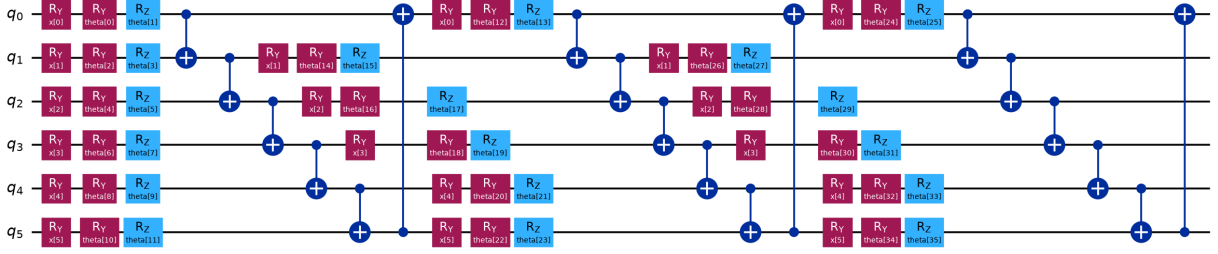


Figure 1: The VQC decision head: 6-qubit, 3-layer data re-uploading ansatz. Each layer re-encodes the CNN features ($R_Y(x)$), applies trainable R_Y/R_Z , and entangles with a CNOT ring. Action values are read out as two-qubit $\langle ZZ \rangle$ correlators.

6. **Readout $\rightarrow Q$ -values.** Three disjoint two-qubit correlators $o_a = \langle Z_{2a} Z_{2a+1} \rangle \in [-1, 1]$ (pairs $(0, 1), (2, 3), (4, 5)$), mapped to $Q_a = \frac{o_a + 1}{2} w_a$ with a trainable output scale w (three params). $\arg \max_a Q_a$ is the action.

Parameter budget (be transparent). Total trainable = 480,339: CNN 480,294 + circuit 36 + $\lambda 6 + w$

3. **The quantum decision head is 0.0075% of the model** (Fig. ??); the CNN does the perception and the VQC is a tiny quantum decision layer. During training the TD-loss gradient flows through $Q \rightarrow \langle ZZ \rangle \rightarrow$ circuit weights $\rightarrow$ encoding $\rightarrow$ CNN in one graph (verified: live gradients on CNN, λ , circuit, w).

3 Training

Model-agnostic Double-QDQN, reusing the backbone verbatim (`src.trainer.dqn.update`, `select_action`, `linear_epsilon`); only the image environment and an image replay buffer are new (the DQN math is not reimplemented). Uniform replay (12,000), batch 32, $\gamma = 0.99$, $\epsilon : 1.0 \rightarrow 0.05$ over 9000 steps, hard target update every 500 gradient steps, one update per 4 env steps, 24,000 decision-steps. Per-group Adam learning rates (CNN 5×10^{-4} , input 3×10^{-3} , variational 10^{-2} , output 3×10^{-2}). Evaluation is greedy ($\epsilon = 0$) over 20 fixed seeds; the 5% exploration floor depresses *training* scores (a stray random action at an obstacle is fatal), so greedy eval is the honest measure.

4 Theory this rests on

- **Data re-uploading** (Pérez-Salinas *et al.*, 2020): re-encoding the input at every layer makes even a small circuit a universal function approximator. We re-upload 3 $\times$; this is why a 36-parameter circuit can represent a useful Q -function at all.
- **Entangled, correlated readout.** The CNOT ring produces a non-separable state; reading two-qubit $\langle Z_i Z_j \rangle$ correlators (rather than single-qubit $\langle Z \rangle$) uses that correlation. §5.2 shows both ingredients are necessary here.
- **Trainability / barren plateaus.** A VQC is a harder optimisation object than a linear layer; we observe exactly this (§5.2), consistent with the barren-plateau literature and the no-clean-advantage findings of Bowles *et al.* (2024).

Agent (identical CNN)	mean	median	best	min
Random policy	224	189	321	—
Scripted oracle (<code>if-else</code>)	1500 [†]	1500	1500	1500
VQC head (quantum)	1565	1547	1875	1434
Classical Linear head	2000[†]	2000	2000	2000

Table 1: Greedy ($\epsilon = 0$) score over 20 seeds; score = frames survived. [†]capped at the 2000-frame episode limit. The quantum head plays well ($7\times$ random, clears dozens of cacti); a classical linear head on the *same* features plays perfectly. No quantum advantage — a small quantum cost.

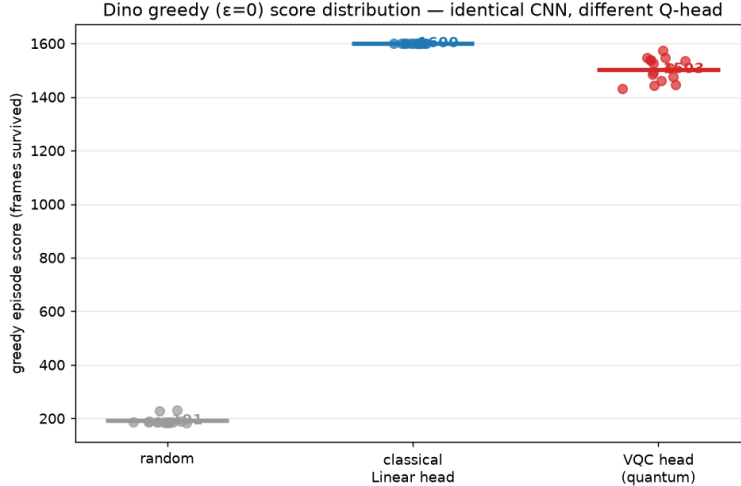


Figure 2: Greedy score distribution. Both heads crush random; classical is perfect and tight, the VQC is strong but slightly lower and noisier (its per-action Q -values are closely spaced).

5 Results

5.1 Main comparison (greedy eval, identical CNN + task)

5.2 Ablations: the quantum structure is load-bearing

Changing exactly one thing from the VQC baseline (Fig. 3):

Variant (one change vs. baseline)	greedy mean	effect
VQC baseline (CNOT ring + $\langle ZZ \rangle$)	1565	—
– remove entangler (separable circuit)	185	collapses to random
– single- Z readout instead of $\langle ZZ \rangle$	185	collapses to random
Frozen ImageNet CNN (variant A)	230	barely above random (out-of-distribution)

Table 2: Single-seed greedy eval. **Removing entanglement OR the two-qubit $\langle ZZ \rangle$ readout each drops the agent from 1565 to 185 (random).** Without entanglement $\langle Z_i Z_j \rangle = \langle Z_i \rangle \langle Z_j \rangle$ factorises; single- Z discards the correlation. Both quantum ingredients are necessary for this agent to function. Variant A (frozen ImageNet CNN) reaches only 230 — ImageNet features are out-of-distribution for a black-and-white line game, as the brief predicted.

5.3 The quantum head deciding, and a harder two-skill task

Figure 4 traces the three $\langle ZZ \rangle$ -derived Q -values over a rollout: flat on an empty track, then the JUMP value spikes at each cactus. A harder *birds-require-duck* mode (opt-in) was added; a classical head learns *both*

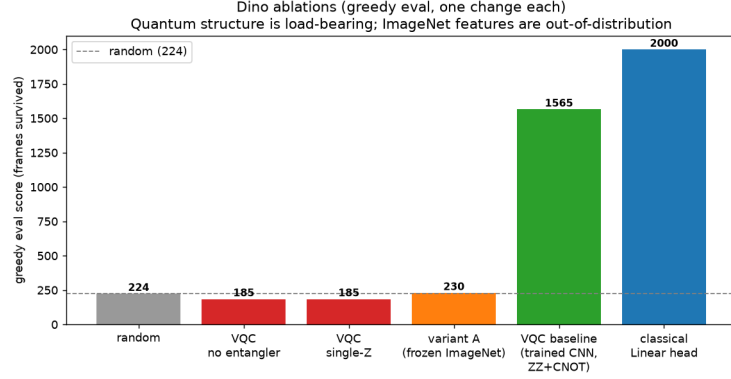


Figure 3: Ablations. The quantum structure does real work: entanglement and the correlated $\langle ZZ \rangle$ readout are each essential.

skills (greedy 1423; per-obstacle it JUMPs cacti 88% and DUCKs birds 75% — opposite responses, i.e. two distinct learned skills). A jump-only policy caps at ~ 640 , so clearing birds genuinely requires the DUCK skill. **The VQC head, however, failed to learn this two-skill task (greedy 185, i.e. random) — in two attempts, including a birds-late/rarer curriculum with 34k steps.** The quantum head masters the one-skill task (cacti, 1565) but not the two-skill task, while the classical head learns both — a robust instance of the trainability gap of §6, not mere under-tuning.

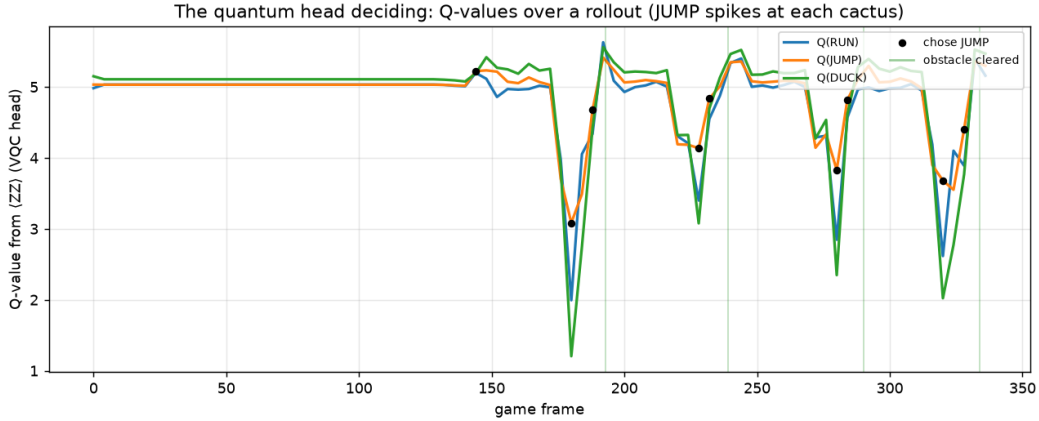


Figure 4: The quantum head’s three action-values over a rollout; black dots mark chosen-JUMP, green lines mark cleared obstacles. The circuit visibly reacts to each cactus.

5.4 It is real, hardware-ready quantum

- **Faithful to Qiskit.** The trained head’s Q -values on `torch_sv` match a genuine Qiskit `EstimatorQNN` (reverse gradient) to 3.4×10^{-7} .
- **Robust to finite shots.** Aer at 1024 shots reproduces the exact action on 5/5 frames; at 128 shots 3/5 (flips only on near-ties).
- **On a real IBM quantum computer** (`ibm_marrakesh`, 156 qubits, 4096 shots, 16 frames): the QPU’s greedy decision matched the exact simulator on **all 16/16 frames**. A device-noise simulation (FakeLagosV2) agreed on 14/16 (mean $|\Delta Q| = 0.070$); the real QPU raw had mean $|\Delta Q| = 0.135$ (Fig. 5). Hardware access uses the saved account’s token via the `ibm_cloud` channel to keep the `venv` at qiskit 1.4.6.

- **Error mitigation helps magnitude, not the decision.** Zero-Noise Extrapolation on the QPU barely changed $|\Delta Q|$ ($0.135 \rightarrow 0.129$) and did *not* improve action-agreement ($16/16 \rightarrow 12/16$) — for a greedy arg max policy the raw device was already sufficient.
- **Compiles to hardware.** Transpiled to the `ibm_lagos` ISA (`{rz, sx, cx}`), the circuit goes from logical depth 27 / 18 CX to **depth 80 / 57 CX** after routing.
- **A whole game on a quantum simulator.** With *every* action decided by a 1024-shot Aer measurement, the agent scored 289 (vs. exact 1200): shot noise flips a near-tie right before an obstacle and ends the run — more shots buy a longer game. A full *live* game on real hardware is impractical ($\sim$ hundreds of *sequential*, non-batchable jobs), but a whole game on Aer is trivial.

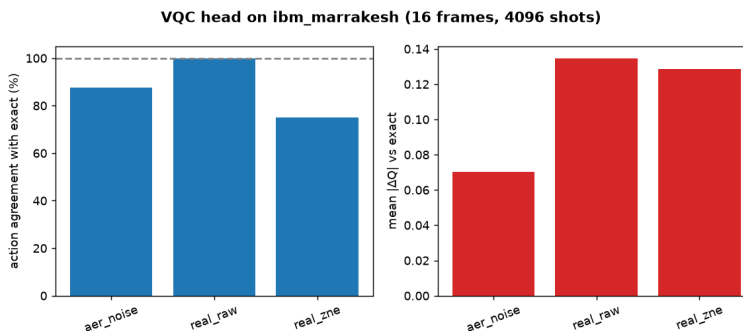


Figure 5: Hardware study on `ibm_marrakesh` (16 frames, 4096 shots): action-agreement with the exact simulator (left) and mean $|\Delta Q|$ (right), for the device-noise simulation and the real QPU with/without ZNE error mitigation. The raw QPU matches all 16 decisions; ZNE lowers magnitude error but not agreement.

6 Honest findings

1. **No quantum advantage** — and none claimed. On identical footing the quantum head is at best comparable to, here slightly weaker than, a classical linear head.
2. **The quantum head is harder to train** than a linear one (needed a higher variational learning rate and more steps); the classical head reached a perfect policy sooner.
3. **But the quantum structure is not decorative:** entanglement and the two-qubit correlated readout are each *necessary* for the agent to work at all (§5.2).
4. **Perception, not the quantum head, set the early ceiling:** the difficulty was reliable jump *timing*, fixed by easing the environment (documented) — standard RL practice that never touches the circuit.

The contribution is the working hybrid pipeline, the honest fair-swap comparison, a visceral pixel-to-qubit demo, and a real-hardware execution — not a benchmark number.

Reproduce

```
python -m experiments.dino.train --head vqc --name dino_vqc --steps 24000 # quantum agent
python -m experiments.dino.train --head classical --name dino_classical --steps 24000
python -m experiments.dino.train --head vqc --entangler none --name dino_abl_noent # ablation
python -m experiments.dino.record_demo --ckpt results/dino_vqc.pt # demo GIF
python -m experiments.dino.visualize_pipeline --ckpt results/dino_vqc.pt # CNN->quantum->env animation
python -m experiments.dino.quantum_readout --ckpt results/dino_vqc.pt # torch_sv == Qiskit (3e-7)
python -m experiments.dino.quantum_play --backend aer --shots 1024 # whole game on Aer
python -m experiments.dino.quantum_hardware --real --backend ibm_marrakesh # real IBM QPU
```